LinXar Listings on Ethereum: Build Guide

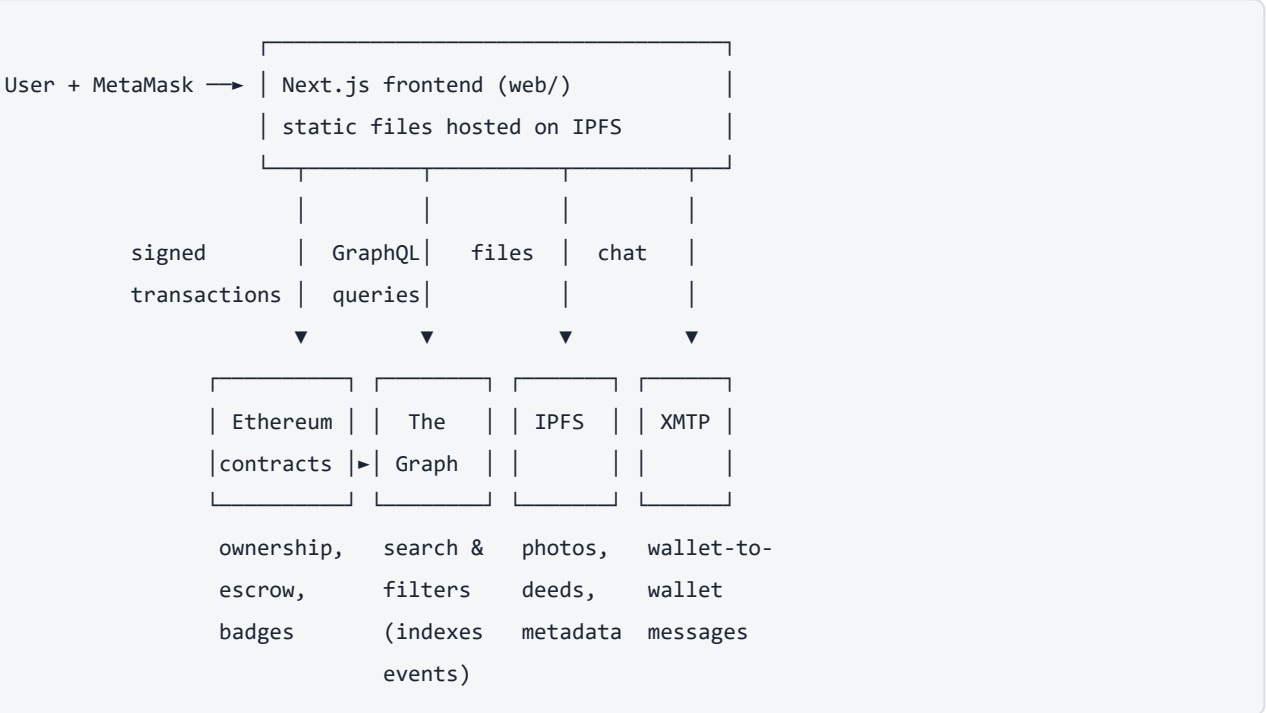
A step-by-step plan for building the LinXar Listings platform (see `Flagship Project_ Property Listing Project.pdf`) as a **full web3 app**: Ethereum smart contracts as the backend, MetaMask for identity, and decentralized services instead of servers and databases. The goal is **learning how blockchain works** along the way.

Work through the milestones in order. Each one adds a real feature and teaches one core concept. Don't move on until the "Done when" checklist passes.

---

1. The big picture

There is **no central server and no database you control**. Every part of the app runs on Ethereum or on a decentralized network:



Part of the app	Where it lives	Why there
Property ownership, escrow, verification badges	**Ethereum contracts**	Needs to be tamper-proof and enforce rules about money
Search, filters, listing feeds	**The Graph** (a decentralized indexer)	Contracts can't search; The Graph reads contract events and serves fast GraphQL queries
Photos, videos, title deeds, listing descriptions	**IPFS**	Far too expensive on-chain; the chain stores only the content hash (CID)
Chat and visit requests	**XMTP** (wallet-to-wallet messaging)	Private, free, and your MetaMask address is your identity; on-chain messages would be public and cost gas
The website itself	**IPFS** (static hosting)	No web server to run or take down
Recommendations, user preferences	**The user's browser**	Computed locally from public data; private preferences never leave the device

What is deliberately *not* stored anywhere public

Personal data such as names, phone numbers and ID numbers. Anything written to Ethereum or IPFS is public and effectively permanent, which conflicts with Ghana's Data Protection Act (people have the right to have their data deleted). Users are identified by wallet address, and contact happens privately over XMTP.

---

2. Safety rules (read before anything else)

1. **Never use real ETH while learning.** Use the local Hardhat chain first, then the **Sepolia** testnet, which has free test ETH.
2. **Create a separate MetaMask wallet for development.** Never import a dev private key into a wallet that holds real money, and never put a real wallet's key in this project.
3. **Never commit private keys or API keys to git.** Use Hardhat's keystore (milestone 8) or a `.env` file listed in `.gitignore`.
4. The test accounts that `npx hardhat node` prints are **publicly known**. Anyone can drain them on a real network. Use them only locally.
5. **Everything on-chain and on IPFS is public forever.** Before writing any data, ask: "Am I OK with anyone in the world reading this, permanently?"

---

3. One-time setup

Already installed on this machine ✅

- Node.js v24, npm 11, Git 2.53, VS Code

Still to do

- ☐ **MetaMask browser extension** from https://metamask.io (only that site; fake extensions are a common scam). Create a new wallet just for development and write the recovery phrase down offline.
- ☐ **VS Code extensions:** "Solidity" by Nomic Foundation, and "GraphQL" (for subgraph schemas and queries).
- ☐ **Accounts you'll need later** (free tiers are enough):
  - Alchemy or Infura: an RPC URL for Sepolia (milestone 8)
  - Etherscan: an API key for verifying contracts (milestone 8)
  - Pinata: IPFS file pinning (milestones 4 and 11)
  - Subgraph Studio: sign in with MetaMask to deploy subgraphs (milestone 9)
  - XMTP needs no account: your wallet *is* your account (milestone 10)

Project layout

Create this structure inside `Land-Listings/`:

```
Land-Listings/
├── BUILD-GUIDE.pdf       ← this guide
├── contracts/            ← Hardhat project: Solidity contracts, tests, deploy scripts
├── subgraph/             ← The Graph: schema, event handlers (milestone 9)
└── web/                  ← Next.js frontend, exported as static files
```

Then initialise git at the `Land-Listings/` level:

```
git init
```

---

4. Milestones

#	Milestone	Main concept
1	First contract (`ListingBoard`)	Contracts, transactions vs. calls, gas
2	Connect MetaMask to a web page	Wallets, signing, ABIs
3	Events and a live feed	Logs and events
4	Files on IPFS, hashes on-chain	Content addressing, what belongs on-chain
5	Properties as NFTs (`PropertyRegistry`)	Token standards, ownership
6	Escrow for deposits (`Escrow`)	Handling money safely, reentrancy
7	Agent verification (`VerifierRegistry`)	Access control, the oracle problem
8	Deploy to Sepolia	Testnets, RPC providers, block explorers
9	Search and filters with The Graph	Decentralized indexing, GraphQL
10	Chat and visit requests with XMTP	Wallet-based messaging, message signing
11	Host the app on IPFS	Fully decentralized frontend

Milestone 1: Your first contract (`ListingBoard`)

Goal: A contract where anyone can post a property listing and anyone can read it.

Steps

1. Create the Hardhat project:

   ```
   mkdir contracts && cd contracts
   npx hardhat --init
   ```

   Pick the **TypeScript + viem** template when asked.
2. Delete the sample contract and create `contracts/ListingBoard.sol` :

   ```
   // SPDX-License-Identifier: MIT
   pragma solidity ^0.8.28;

   contract ListingBoard {
       struct Listing {
           uint256 id;
           address owner;      // who posted it (msg.sender)
           string title;
           string location;
           uint256 priceWei;   // price in wei (1 ETH = 10^18 wei)
   ```

```
        bool active;
    }

    Listing[] private listings;   // stored permanently on-chain

    // Changes state, so it costs gas and needs a signed transaction
    function createListing(string calldata title, string calldata location, uint256 priceWei)
        external
        returns (uint256 id)
    {
        id = listings.length;
        listings.push(Listing(id, msg.sender, title, location, priceWei, true));
    }

    // Only the owner may deactivate their listing
    function deactivateListing(uint256 id) external {
        require(listings[id].owner == msg.sender, "Not your listing");
        listings[id].active = false;
    }

    // "view" functions only read data: free, no transaction needed
    function getListing(uint256 id) external view returns (Listing memory) {
        return listings[id];
    }

    function totalListings() external view returns (uint256) {
        return listings.length;
    }
}
```

3. Compile: `npx hardhat compile`
4. Write tests in `test/` covering: creating a listing, reading it back, and checking that a *different* account **cannot** deactivate it. Run them with `npx hardhat test`.
5. Start a local blockchain in its own terminal: `npx hardhat node`. Leave it running and look at the 20 funded test accounts it prints.
6. Deploy with Hardhat Ignition (write a module in `ignition/modules/`), then run: `npx hardhat ignition deploy ignition/modules/ListingBoard.ts --network localhost` Note the **contract address** it prints.

Concepts to understand

- A **contract** is code plus storage that lives at an address on the chain.
- **Transactions** (state changes) cost **gas** and must be signed. **Calls** to `view` functions are free.
- `msg.sender` is the address that signed the transaction. This is how the contract knows who you are; there are no usernames or passwords.
- `require` reverts the whole transaction if the condition fails, so nothing changes (but gas is still spent).

Done when

- ☐ All tests pass, including the "not your listing" test
- ☐ The contract is deployed to the local node and you have its address
- ☐ You can explain the difference between a transaction and a call

---

Milestone 2: Connect MetaMask to a web page

Goal: A Next.js page with a "Connect Wallet" button that shows listings and lets you post a new one, signed in MetaMask.

Steps

1. Add the local chain to MetaMask (Settings → Networks → Add network manually):
   - Network name: `Hardhat Local`
   - RPC URL: `http://127.0.0.1:8545`
   - Chain ID: `31337`
   - Currency symbol: `ETH`
2. Import one of the private keys that `npx hardhat node` printed (MetaMask → Add account → Import account). You should now see 10,000 test ETH.
3. Create the frontend from `Land-Listings/`:

   ```
   npx create-next-app@latest web
   cd web
   npm install viem
   ```

4. Copy the contract's **ABI** from `contracts/artifacts/contracts/ListingBoard.sol/ListingBoard.json` into the web app, along with the deployed address.
5. Build it by hand first (it teaches more than a library would):
   - Detect MetaMask through `window.ethereum`.
   - Request accounts with `eth_requestAccounts`, then show the connected address.
   - Check the chain ID is `31337`, and ask the user to switch networks if it isn't.
   - Read `totalListings()` and `getListing(i)` with a viem **public client**.
   - Send `createListing(...)` with a viem **wallet client**. MetaMask will pop up asking you to sign.
   - Wait for the transaction receipt, then refresh the list.
6. Optional, later: switch to **wagmi** (React hooks built on viem) once you understand what it does underneath.

> 💡 Since the app will be hosted as static files (milestone 11), keep all blockchain logic in the browser. Don't use Next.js API routes or server-side code.

Concepts to understand

- Your **account** is a key pair. The address comes from the public key, and MetaMask holds the private key.
- MetaMask never sends your private key anywhere. It **signs** the transaction locally and sends the signed result.
- The **ABI** is the contract's interface: how the frontend knows which functions exist and how to encode calls to them.
- A transaction goes through several states: *pending*, then *mined*, then *confirmed*.

Done when

- ☐ You can connect, post a listing from the browser, sign it in MetaMask, and see it appear
- ☐ You've seen the gas fee in the MetaMask popup and noticed your ETH balance drop
- ☐ Switching MetaMask to a different account shows a different `owner` on new listings

---

Milestone 3: Events and a live feed

Goal: New listings appear on the page automatically, without a refresh.

Steps

1. Add an event to `ListingBoard`:

   ```
   event ListingCreated(uint256 indexed id, address indexed owner, string title, uint256 priceWei);
   ```

   and `emit ListingCreated(...)` inside `createListing`.
2. Add a test that checks the event is emitted with the correct values.
3. Redeploy, then update the ABI and address in `web/`.
4. In the frontend, use viem's `watchContractEvent` to listen for `ListingCreated`, and `getContractEvents` to load past events.

Concepts to understand

- **Events** are written to transaction **logs**. They are cheaper than storage, but contracts themselves can't read them back.
- `indexed` parameters can be filtered on (for example, "all listings by owner X").
- This is how real dApps build feeds and histories: they read events, not storage. In milestone 9, The Graph will do this for you at scale.

Done when

- ☐ Posting from one browser tab makes the listing appear in another tab automatically

---

Milestone 4: Files on IPFS, hashes on-chain

Goal: Listings have photos and documents without storing them on-chain.

Steps

1. Create a Pinata account and get an API key (JWT).
2. Upload images to Pinata. You'll get back a **CID** (content ID) for each file.
3. Create a **metadata JSON** file per listing (title, description, photo CIDs, amenities, latitude/longitude) and upload that too.
4. Change the contract so each listing stores:
   - the `metadataCID` (pointing to the JSON on IPFS), and
   - a few **filterable fields** on-chain and in the event: `category` (Residential, Commercial, Industrial, Land), `region` (e.g. "Greater Accra"), `priceWei`.

   Long text and media go to IPFS; the fields people will *filter* on go in events so The Graph can index them (milestone 9).
5. The frontend fetches the JSON from an IPFS gateway and displays it.

> ⚠️ **The one centralized compromise.** The Pinata key must stay secret, but a static app has no server to hide it on. Options:
>
> - **While learning locally:** keep the key in `web/.env.local` (git-ignored) and upload from a script on your own machine.
> - **For public users:** a tiny serverless function (for example, on Cloudflare Workers) whose only job is to hand out short-lived, signed upload URLs from Pinata. It never touches listings or users, and the app keeps working for reading even if it goes down.
>
> Never put the key in browser code.

Concepts to understand

- On-chain storage costs a lot: about 20,000 gas for every 32 bytes. A single photo would cost a fortune.
- A **CID** is derived from the file's contents. If the file changes, the CID changes, so the on-chain CID proves the documents haven't been tampered with.
- **Pinning:** IPFS nodes only keep files someone has asked them to keep. Pinata "pins" yours so they stay available.

Done when

- ☐ A listing shows a photo loaded from IPFS
- ☐ You can explain why changing the photo would require a new on-chain transaction

---

Milestone 5: Properties as NFTs (`PropertyRegistry`)

Goal: Each registered property is a unique token. Owning the token means owning the on-chain record.

Steps

1. Install OpenZeppelin in `contracts/`: `npm install @openzeppelin/contracts`
2. Create `PropertyRegistry.sol`, extending `ERC721`:
   - `registerProperty(address owner, string metadataCID, bytes32 deedHash)` mints a token
   - `tokenURI` returns `ipfs://<CID>`
   - Keep a `mapping(bytes32 => bool)` of registered deed hashes, and **reject a deed that's already registered**. This blocks the common Ghanaian fraud of selling the same plot to several buyers, and it's enforced by the contract, not by a company.
3. Test minting, transferring, ownership lookups, and the duplicate-deed rejection.
4. In the frontend, add a "My Properties" page that shows the tokens the connected wallet owns.

Concepts to understand

- **Token standards** (ERC-721 for unique items, ERC-20 for currencies) are agreed interfaces, which is why wallets and marketplaces work with any token.
- **OpenZeppelin** provides audited, standard implementations. Don't write your own token logic from scratch.
- Transferring the NFT transfers the on-chain ownership record, and the `Transfer` events form a permanent ownership history.
- `bytes32` hashes let you prove a document exists without publishing the document itself.

Done when

- ☐ You can register a property, transfer it to another account, and see the new owner in the UI
- ☐ Registering the same deed twice reverts

Milestone 6: Escrow for deposits (`Escrow`)

Goal: A buyer or renter deposits ETH into a contract instead of handing it to an agent. Funds are released only when the agreed conditions are met.

Design to implement

- `openDeal(seller, propertyId)`: buyer sends ETH (`payable`)
- `confirm()`: both sides confirm, then the seller can withdraw
- `requestRefund()` / `raiseDispute()`: covers the brief's "dispute resolution tools"
- `resolveDispute()`: callable only by an arbiter role
- A deadline after which the buyer can reclaim the funds if nothing happened

Security patterns (essential)

- **Checks → Effects → Interactions:** update balances *before* sending ETH.

- Use **pull payments**: users call `withdraw()`, rather than the contract pushing ETH to them.
- Add OpenZeppelin's `ReentrancyGuard` to functions that send ETH.
- Write tests for every bad path: double withdrawal, a stranger calling `confirm`, refund after release, and so on.

Concepts to understand

- `payable` functions can receive ETH, and `msg.value` is how much was sent.
- **Reentrancy:** a malicious contract can call back into yours before your function finishes. This is what caused the 2016 DAO hack.
- Bugs in money contracts can't be patched once deployed. That's why tests matter so much here.

Done when

- ☐ Full deal flow works in the UI with two MetaMask accounts (buyer and seller)
- ☐ Tests cover every failure case listed above

Milestone 7: Agent verification (`VerifierRegistry`)

Goal: A "Verified" badge for agents and sellers (from the brief), with only trusted verifiers able to register properties.

Steps

1. Use OpenZeppelin `AccessControl` with roles such as `ADMIN_ROLE` and `VERIFIER_ROLE`.
2. Verifiers can mark an agent address as verified (and revoke it).
3. Restrict `PropertyRegistry.registerProperty` so only verifiers can call it.
4. Show the verified badge in the UI by reading the contract.

Concepts to understand

- **Access control:** contracts are public, so anyone can call any function unless you restrict it.
- **The oracle problem:** the chain only knows what someone tells it. If a verifier approves a fake land title, the chain records the lie permanently. Blockchain gives you a tamper-proof *record*, not proof that the record is *true*. In practice, verifiers would need to be trusted parties such as the Lands Commission or licensed surveyors.

Done when

- ☐ A non-verifier account gets a revert when trying to register a property
- ☐ You can explain what blockchain *can't* guarantee about land ownership

---

Milestone 8: Deploy to the Sepolia testnet

Goal: Your contracts run on a public network that anyone can inspect.

Steps

1. Get a Sepolia RPC URL from Alchemy or Infura.
2. Get free Sepolia ETH from a faucet (for example, the Google Cloud Web3 faucet or Alchemy's faucet).
3. Store secrets in Hardhat's encrypted keystore, not in code:

   ```
   npx hardhat keystore set SEPOLIA_RPC_URL
   npx hardhat keystore set SEPOLIA_PRIVATE_KEY
   npx hardhat keystore set ETHERSCAN_API_KEY
   ```

   Then reference them in `hardhat.config.ts` with `configVariable("SEPOLIA_RPC_URL")` and so on. Use your **dev wallet's** key only.
4. Deploy: `npx hardhat ignition deploy ignition/modules/<Module>.ts --network sepolia`
5. Verify the source on Etherscan so anyone can read your contract code (Ignition supports a `--verify` flag).
6. **Write down each contract's address and the block number it was deployed in.** The Graph needs both in milestone 9.
7. Point `web/` at the Sepolia addresses and switch MetaMask to Sepolia.

Concepts to understand

- **RPC providers** are the nodes your app talks to. You don't need to run your own.
- **Block explorers** (sepolia.etherscan.io) show every transaction, event and storage change publicly.
- **Verifying** the source proves that the deployed bytecode matches your published code.

Done when

- ☐ You can look up your contract on sepolia.etherscan.io and read its verified code
- ☐ A friend with MetaMask on Sepolia can use your app

Milestone 9: Search and filters with The Graph

Goal: Fast search and filtering (location, price, category) without running your own database. This replaces a traditional backend.

How it works: a **subgraph** is a set of instructions telling The Graph's indexers which contract events to watch and how to turn them into searchable records. The indexers process every event and answer GraphQL queries from your frontend.

Steps

1. Install the Graph CLI and create a subgraph in Subgraph Studio (sign in with MetaMask):

   ```
   npm install -g @graphprotocol/graph-cli
   cd Land-Listings
   ```

```
graph init          # choose Ethereum → Sepolia, paste your contract address
```

Move the generated project into `subgraph/` if needed. Set `startBlock` in `subgraph.yaml` to your deployment block, so indexing doesn't scan the whole chain.

2. Define your data in `schema.graphql`:

```
type Listing @entity(immutable: false) {
  id: ID!
  owner: Bytes!
  category: String!
  region: String!
  priceWei: BigInt!
  metadataCID: String!
  active: Boolean!
  createdAt: BigInt!
}
```

3. Write **event handlers** in `src/` (AssemblyScript, a TypeScript-like language). Each one turns an event into an entity:

```
export function handleListingCreated(event: ListingCreated): void {
  let listing = new Listing(event.params.id.toString())
  listing.owner = event.params.owner
  listing.category = event.params.category
  listing.region = event.params.region
  listing.priceWei = event.params.priceWei
  listing.metadataCID = event.params.metadataCID
  listing.active = true
  listing.createdAt = event.block.timestamp
  listing.save()
}
```

Add handlers for deactivation, NFT `Transfer`s, escrow deals and verification changes too.

4. Build and deploy:

```
graph codegen && graph build
graph auth <your deploy key from Subgraph Studio>
graph deploy <your-subgraph-name>
```

5. Query it from the frontend (Studio gives you the query URL). For example, active land listings in Greater Accra under 5 ETH, cheapest first:

```
{
  listings(
    where: { active: true, category: "Land", region: "Greater Accra",
             priceWei_lt: "500000000000000000" }
    orderBy: priceWei
  ) { id owner priceWei metadataCID }
}
```

6. **Maps:** show the results on a Mapbox map using the latitude/longitude from each listing's IPFS metadata. Mapbox *public* tokens (`pk.…`) are designed to be used in browsers. Restrict yours to your site's URL in the Mapbox dashboard.

Concepts to understand

- **Indexing:** contracts are bad at queries, so dApps read their events into a searchable store. The Graph does this on a decentralized network of indexers instead of your own server.
- **Design for indexing:** only data that appears in events can be searched. This is why milestone 4 put category, region and price in the event.
- **Eventual consistency:** a new listing appears in search a few seconds after its transaction is mined, once the indexers have processed the block.

Done when

- ☐ The search page filters listings by category, region and price using The Graph
- ☐ A new listing shows up in search shortly after it's posted
- ☐ Listings appear as pins on a map

Milestone 10: Chat and visit requests with XMTP

Goal: Buyers can message agents and request property visits privately, wallet-to-wallet, with no chat server of your own (the brief's "instant messaging" and "appointment scheduling").

Steps

1. Install the XMTP browser SDK in `web/`: `npm install @xmtp/browser-sdk`
2. Create an XMTP client using MetaMask as the **signer**. MetaMask asks the user to sign a message (not a transaction, so no gas) to create their messaging identity. Use XMTP's `dev` network while learning.
3. On a listing page, add a "Message agent" button that opens a conversation with the listing owner's address. Check first that the address can receive XMTP messages (`canMessage`). If it can't, show "This agent hasn't enabled messaging yet."
4. **Visit requests:** send a structured message (for example, JSON with proposed dates) that the agent can accept or decline in the chat UI. Optionally, record a confirmed visit on-chain only if it needs to be provable later.
5. Add an inbox page listing all of the user's conversations.

Concepts to understand

- **Signing a message vs. sending a transaction:** both prove you control an address, but only a transaction changes the chain and costs gas. Signed messages are how web3 apps do "login".
- **End-to-end encryption:** XMTP messages are encrypted so only the sender and recipient can read them. This is why chat belongs here rather than on Ethereum, where everything is public.
- **Wallet as identity:** the same MetaMask address owns properties, signs escrow deals and receives messages. There's no separate account system.

Done when

- ☐ Two MetaMask accounts can chat about a listing in two different browsers
- ☐ A visit request can be sent and accepted in the chat
- ☐ You can explain why chat isn't stored on Ethereum

Milestone 11: Host the app on IPFS

Goal: The website itself is decentralized, so there's no web server that can go down or be taken offline.

Steps

1. Configure Next.js for a static export in `web/next.config.ts`:

   ```
   const nextConfig = { output: "export", images: { unoptimized: true } };
   export default nextConfig;
   ```

   This fails to build if anything still uses server-side features, which is a good check that the app is fully client-side.
2. Build: `npm run build`. The site is generated into `web/out/`.
3. Upload the `out/` folder to IPFS (through Pinata or a hosting service such as Fleek). You get one CID for the whole site.
4. Open it through an IPFS gateway (for example, `https://<gateway>/ipfs/<CID>/`) and test every page.
5. Optional: register an **ENS name** (Ethereum Name Service) and set its `contenthash` record to your site's CID, so the app has a readable address that points to the decentralized site. Practise on Sepolia first; ENS on mainnet costs real ETH.

Concepts to understand

- **Immutable deployments:** each build gets a new CID. Updating the site means publishing a new CID and pointing your ENS name at it.
- **Gateways** let ordinary browsers load IPFS content over HTTPS.
- At this point, no part of LinXar depends on a server you run (apart from the optional upload helper in milestone 4).

Done when

- ☐ The full app works when loaded from an IPFS gateway URL
- ☐ You can explain what would still work if your own computer was switched off forever

5. What about the AI features?

The brief asks for AI recommendations and fraud detection. Machine-learning models can't run on Ethereum, and a full web3 app has no server to run them on, so they're redesigned:

Recommendations: computed in the user's browser

- The frontend saves what the user views and likes in the browser's local storage. It never leaves their device, so there's no privacy problem.
- It scores listings from The Graph against those preferences (category, region, price range, amenities) and shows the best matches.
- A simple scoring formula is enough to start with. Later, small models can run in the browser using TensorFlow.js, which matches the brief's TensorFlow tooling.

Fraud detection: rules enforced by contracts, plus checks on public data

- **Prevented by contracts:** duplicate deed registration (milestone 5), unverified agents registering properties (milestone 7), and deposits disappearing (escrow, milestone 6).
- **Flagged in the frontend using The Graph:** listings priced far below others in the same region, brand-new wallets posting many listings, properties transferred several times in a short period. Show these as warnings to buyers.

> Heavier AI, such as image analysis to spot reused property photos, needs real computing power. That would mean either an off-chain service (and a step back from full web3) or decentralized compute networks, which are still immature. Leave it until the rest works.

6. Troubleshooting

Problem	Fix
MetaMask says "nonce too high" or transactions hang after restarting `npx hardhat node`	The local chain was reset but MetaMask remembers old transactions. Go to MetaMask Settings → Advanced → **Clear activity tab data**.
Frontend calls fail after redeploying	Redeploying creates a **new address**. Update the address (and the ABI, if the contract changed) in `web/` and in `subgraph/subgraph.yaml`.
"Transaction reverted" with no reason	Check the `require` messages in your contract, and run the same action in a test to see the revert reason.
MetaMask is on the wrong network	Check the chain ID in your frontend and prompt the user to switch (`wallet_switchEthereumChain`).
Faucet won't give Sepolia ETH	Some faucets require a small mainnet balance. Try a different faucet.
Subgraph shows no data	Check the contract address, the network (`sepolia`) and `startBlock` in `subgraph.yaml`. Look at the indexing logs in Subgraph Studio for handler errors.
Subgraph fails after changing a contract	Run `graph codegen` again so the generated types match the new ABI, then rebuild and redeploy.
XMTP says an address can't be messaged	That wallet hasn't created an XMTP identity yet. They need to open the app and connect once.
Static export build fails	Something still uses server-side Next.js features (API routes, server actions, dynamic server rendering). Move that logic to the browser.

7. Glossary

- **Address:** a 20-byte account identifier (0x...), derived from a public key.
- **ABI:** the JSON description of a contract's functions and events.
- **Gas:** units of computation. Fee = gas used × gas price.
- **Wei:** the smallest unit of ETH. 1 ETH = 10^{18} wei.
- **Transaction:** a signed instruction that changes chain state.
- **Call:** a free, read-only query.
- **Signed message:** proof that you control an address, without changing the chain or paying gas.
- **Event / log:** data emitted by a contract, readable by apps.
- **Testnet:** a public chain with worthless ETH, for testing (Sepolia).
- **CID:** an IPFS content identifier, derived from the file's contents.
- **Pinning:** asking an IPFS service to keep your files available.
- **Gateway:** a website that serves IPFS content to ordinary browsers.
- **NFT (ERC-721):** a unique token with a single owner.

- **Reentrancy:** an attack where an external call re-enters your contract before it finishes.
- **Subgraph:** a definition telling The Graph which events to index and how to store them.
- **GraphQL:** the query language used to search a subgraph.
- **XMTP:** an encrypted messaging network where wallet addresses are the identities.
- **ENS:** Ethereum Name Service, readable names (like `linxar.eth`) for addresses and websites.

---

8. Learning resources

- Ethereum developer docs: https://ethereum.org/developers
- Solidity docs: https://docs.soliditylang.org
- Hardhat docs: https://hardhat.org/docs
- viem docs: https://viem.sh
- OpenZeppelin contracts: https://docs.openzeppelin.com/contracts
- The Graph docs: https://thegraph.com/docs
- XMTP docs: https://docs.xmtp.org
- IPFS docs: https://docs.ipfs.tech
- ENS docs: https://docs.ens.domains
- CryptoZombies (interactive Solidity tutorial): https://cryptozombies.io
- Ethernaut (security challenges, to try after milestone 6): https://ethernaut.openzeppelin.com